

Embedded Systems and IoT

Ingegneria e Scienze Informatiche - UNIBO

a.a 2025/2026

Prof. Alessandro Ricci

[Lab Notes]

MCU PROGRAMMING BASICS

OBJECTIVE

- This lab module provides a practical overview of main aspects related to microcontroller programming, based on Arduino and Wiring
- In particular:
 - digital I/O
 - output PWM
 - analog input
 - interrupt
 - timer
 - serial TTL

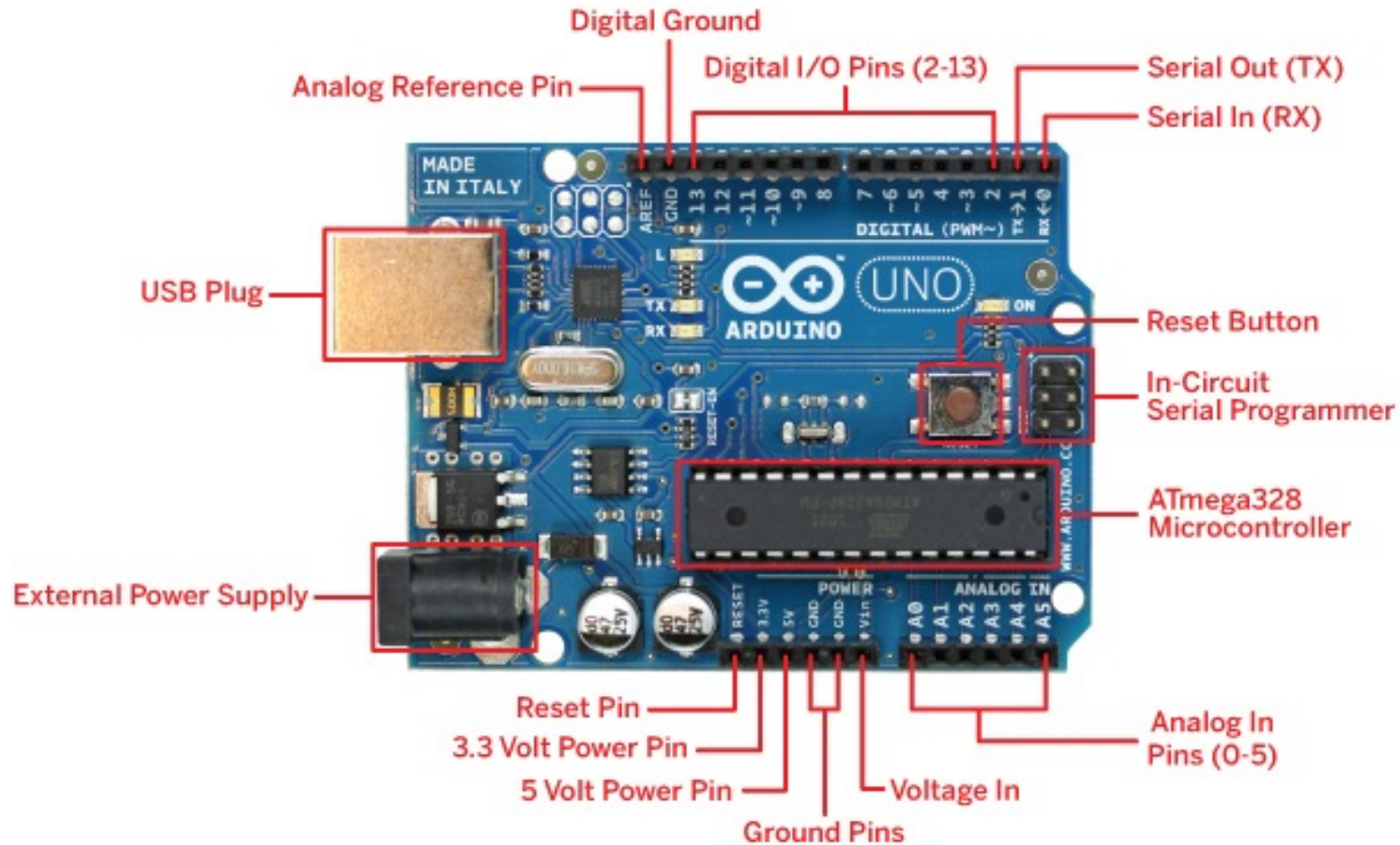
ARDUINO - INTRO

- The Arduino project (<https://en.wikipedia.org/wiki/Arduino>)
 - ...*The Arduino project began in 2005 as a tool for students at the [Interaction Design Institute Ivrea](#), Italy, aiming to provide a low-cost and easy way for novices and professionals to create devices that interact with their environment using [sensors](#) and [actuators](#)...*
...*The Arduino project was started at the [Interaction Design Institute Ivrea](#) (IDII) in [Ivrea](#), Italy.*
[2] At that time, the students used a [BASIC Stamp microcontroller](#) at a cost of \$50. In 2003 [Hernando Barragán](#) created the development platform [Wiring](#) as a Master's thesis project at IDII, under the supervision of Massimo Banzi and [Casey Reas](#). Casey Reas is known for co-creating, with Ben Fry, the [Processing](#) development platform. The project goal was to create simple, low cost tools for creating digital projects by non-engineers. The Wiring platform consisted of a [printed circuit board](#) (PCB) with an [ATmega128](#) microcontroller, an IDE based on Processing and library functions to easily program the microcontroller.^[4] In 2005, Massimo Banzi, with David Mellis, another IDII student, and David Cuartielles, extended Wiring by adding support for the cheaper ATmega8 microcontroller. The new project, forked from Wiring, was called Arduino.
 - *team: Massimo Banzi, David Cuartielles, Tom Igoe, Gianluca Martino, e David Mellis*
- **Open-source**
 - hardware and software
- **Makers** vision - prototyping, tinkering, inventing
- Web site: <http://arduino.cc>

ARDUINO - TEAM (PRE 2015)



IN LAB: ARDUINO UNO



ARDUINO UNO

- Main components
 - MCU ATmega 328P
 - 8 bit, 16 MHz
 - Flash memory: 32 KB
 - SRAM memory: 2 KB, EEPROM: 1 KB
 - 14 digital input/output pins
 - 6 can be used as PWM output
 - 6 analog input
 - USB connector
 - power jack
 - ICSP header
 - reset button

Further specs

Operating voltage: 5V

Input voltage (recom.): 7-12 V

DC current per I/O pin: 40 mA

DC current per 3.3V: 50mA

<https://docs.arduino.cc/hardware/uno-rev3>

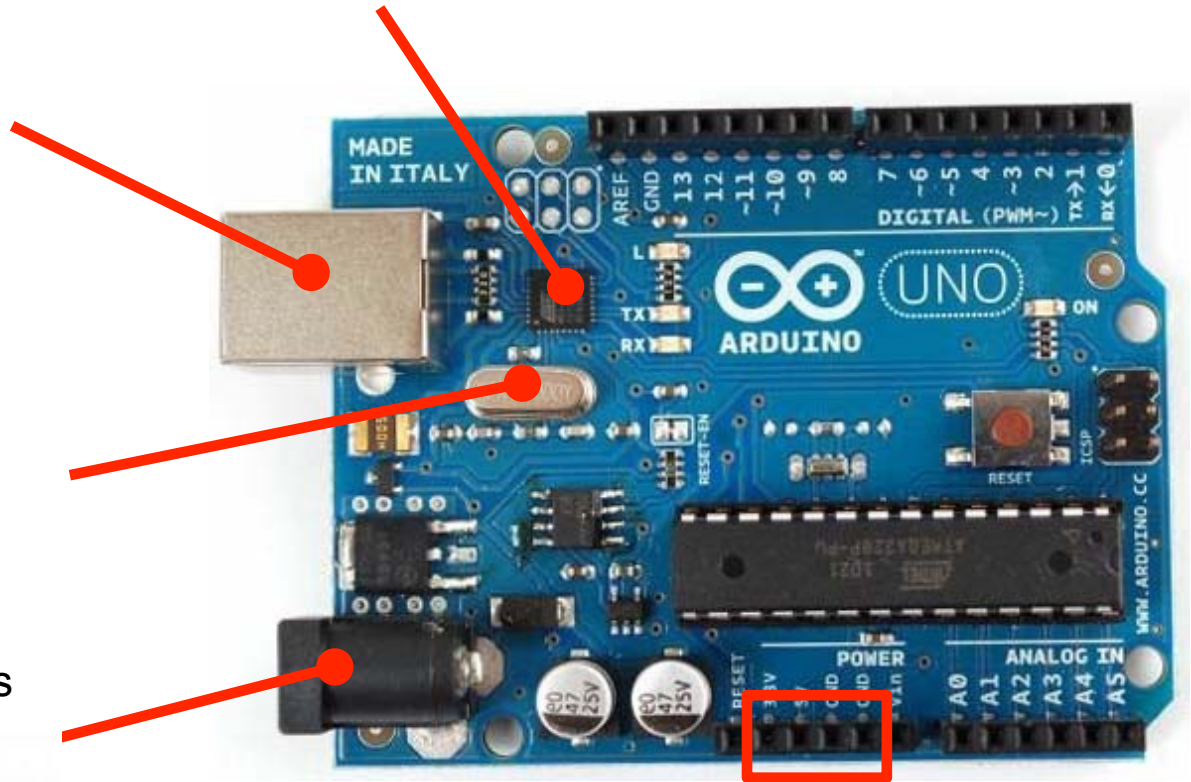
ARDUINO UNO

μ C to convert serial TTL into USB

PC connection via USB
(power + serial)

Quartz oscillator for the
16 MHz clock

External power/batteries
(6-20 Volt)



Pins that provide the reference voltage for
circuits: GND, 5 V (3.3V also available)

ARDUINO UNO

Onboard led, connected to pin 13

14 line strip connector => 14 GPIO pins

power on
led

reset
button

ICSP
connector

Led signalling TX and RX
activities (serial)



6 analog input strip connector

PIN SPECS

- **Digital pins 0..13**
 - nominal voltage: 5V
 - can supply or draw current up to 40mA
- **Analog input A0...A5**
 - nominal voltage: 5V
 - 10-bit resolution
 - 0-5 Volt value => converted by the ADC into a 10-bit value [0..1023 ($2^{10}-1$)]
 - AREF pin (pin 21) makes it possible to change the reference value (max value) for the ADC

EXTERNAL POWER

- Through the PC via USB
 - Through batteries or a power supply
 - battery capacity depends on desired battery life, the current drawn
 - Example:
 - If we have an estimated power consumption of 150mA
 - ..and we want a battery life of 8 hours
- => then the battery capacity should be $\geq 150\text{mA} \times 8 = 1200\text{mAh}$

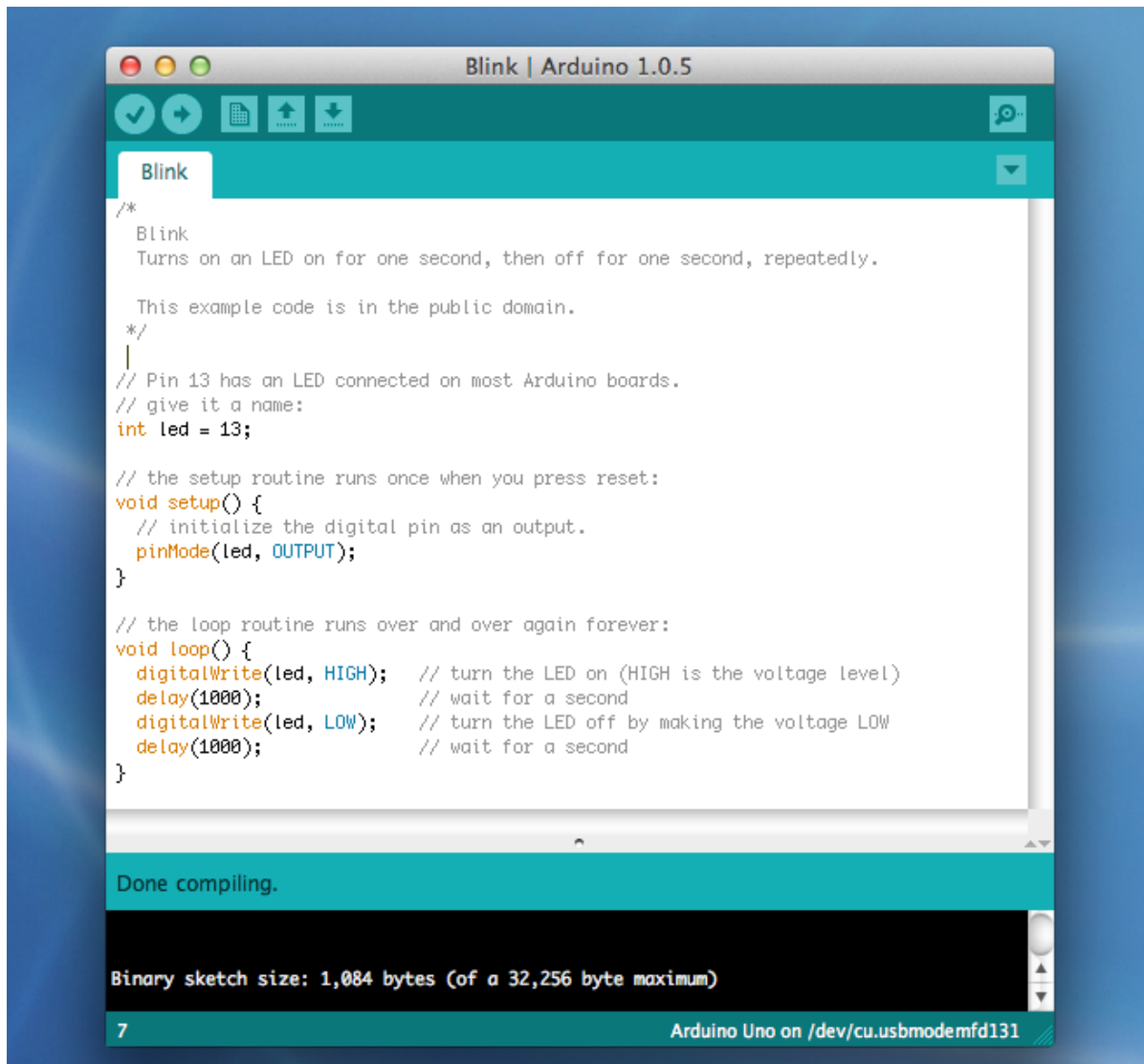
PROGRAMMING

- Reference framework used here: **Wiring**
 - open-source framework for microcontrollers
 - cross-platform, numerose piattaforme supportate - AVR Xmega, AVR Tiny, TI MSP430, Microchip PIC24/32...
 - based on **GNU GCC**, with a set of custom libraries
 - C/C++ language
 - AVR gcc tool chain - avr-gcc compiler
 - libc customized to access and interact with the HW
 - <http://www.nongnu.org/avr-libc/>
 - large documentation and community
 - <http://wiring.org.co/learning/tutorials/>
 - link: <http://arduino.cc/en/Reference/HomePage>, <http://wiring.org.co/>
- Directly supported by **Arduino IDE**
- Large set of available libraries
 - <https://www.arduino.cc/reference/en/libraries/>

IDE AND TOOLS

- Every Integrated Development Environment is meant to be installed on some host PC, connected to Arduino via USB
- **Arduino IDE**
 - very simple and effective for small-size projects
 - based on Wiring framework
 - implemented in Java
 - <http://arduino.cc/en/Main/Software>
- **PlatformIO and Visual Studio Code**
 - for more complex projects
 - PlatformIO - <https://platformio.org/>
 - Visual Studio code: <https://code.visualstudio.com/>
 - Arduino plugin extension to be installed
 - PlatformIO extension
- **Simulators/Emulators**
 - <https://wokwi.com/>
 - An open-source one: Tinkercad - <https://www.tinkercad.com/>

ARDUINO IDE



WIRING PROGRAM STRUCTURE: SUPER LOOP

- Wiring adopts the **superloop** as control architecture
- Two main procedures
 - **setup()**
 - instructions to be executed at the program boot, to setup and configure the initial state
 - executed once
 - **loop()**
 - body of the control loop
 - executed repeatedly

PROGRAM UPLOAD AND EXECUTION

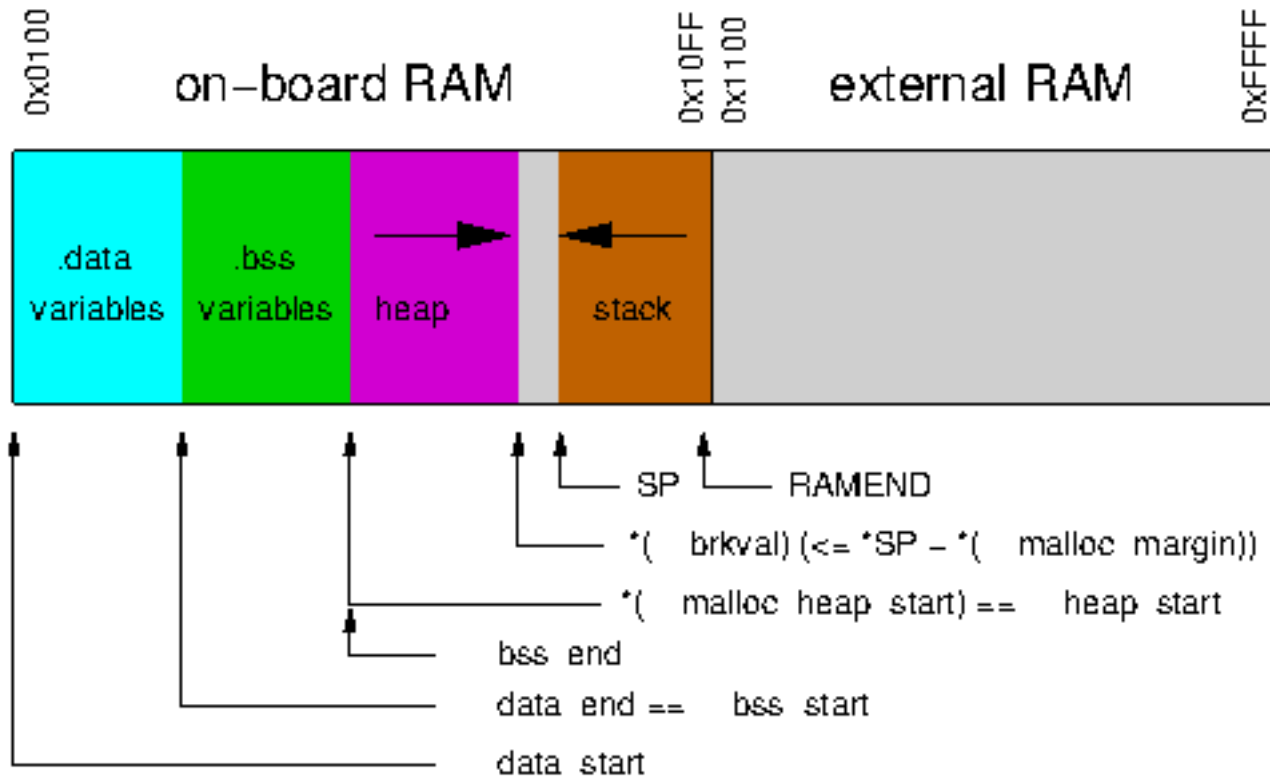
- Two ways to upload a program on Arduino
 - directly by a connected PC, exploiting the **bootloader** preloaded on the MCU
 - <http://arduino.cc/en/Tutorial/Bootloader>
 - using an external In-System Programmer device, that allows to program the microcontroller through the ICSP (In-Circuit-Serial-Programming) interface
 - <http://arduino.cc/en/Main/ArduinoISP>
- We are going to use the first one.

BOOTING - DETAILS

- Once Arduino is reset (either through the button, or the pin, or USB), the bootloader program is executed
- If the bootloader receives immediately a proper command by the IDE (via serial), then it tries to upload the binary sent by the IDE
 - the binary (executable) is downloaded in memory (FLASH) and then executed
 - this occurs each time we select UPLOAD from the IDE => a reset signal is sent to Arduino, starting then the upload protocol
- If the bootloader does not receive any command, then the last executable in memory is executed

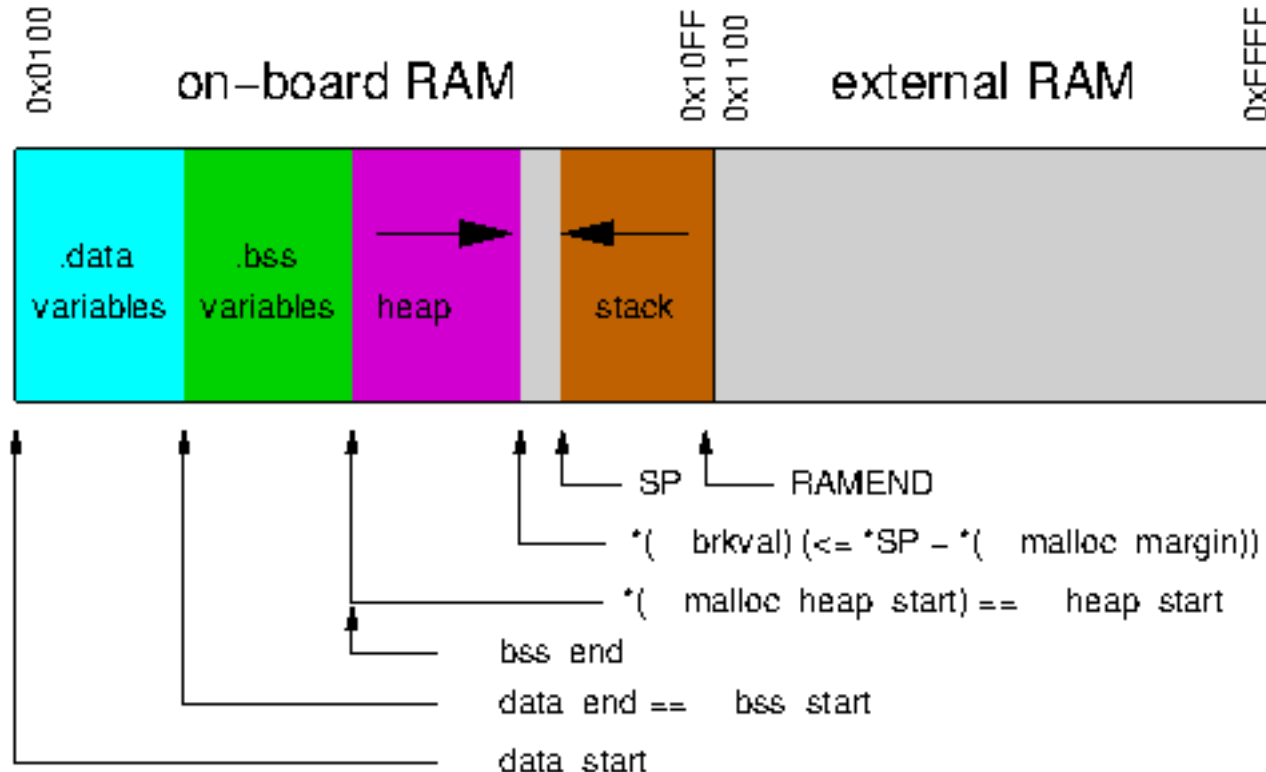
RUNTIME - MEMORY MAP

- Memory map made by the compiler avr-gcc / segments:
 - .data (global vars), .bss (constants), heap, stack
- Note
 - heap and stack grows in opposite directions
 - can collide in the case of memory overflow



RUNTIME - MEMORY MAP

- Internal registers (including I/O, timer..) are mapped onto 0..0xFF address space
 - so the first useful memory address is 0x100 (\$100) i.e. 256 and the last is 0x8FF (\$8FF) i.e. 2303 (256 + 2KiB)

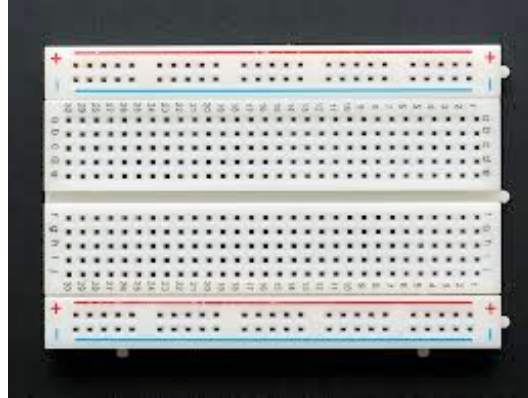


LAB WORK ENVIRONMENT SETUP

- Hardware
 - host computer + Arduino UNO + cable
 - basic electronic components
 - breadboard
 - resistors: 220 Ω , 1K Ω , 10 K Ω
 - led, tactile button, potentiometer
- Software
 - **Arduino IDE**
 - framework Wiring
 - to draw circuit parts:
 - **Tinkercad (open source)**
 - Eagle, *Fritzing*
- Shared instruments
 - tester (multimeter)

MATERIAL: THE BREADBOARD

- Useful for creating circuits without soldering
 - <https://en.wikipedia.org/wiki/Breadboard>

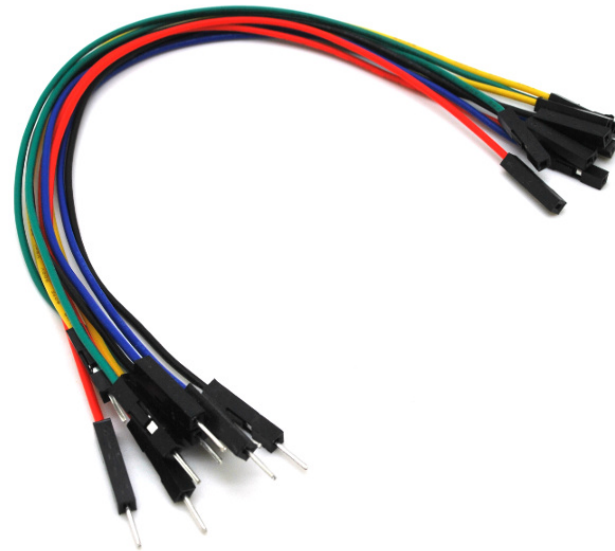


- Main points
 - terminal strips - the main areas, to hold most of the electronic components.
 - In the middle of a terminal strip of a breadboard, one typically finds a notch running in parallel to the long side. The notch is to mark the centerline of the terminal strip. The clips on the right and left of the notch are each connected in a radial way; typically five clips (i.e., beneath five holes) in a row on each side of the notch are electrically connected.
 - Bus strips - To provide power to the electronic components
 - connected horizontally

JUMPERS



(male-male)



(male-female)

WARNING

- **Avoid short circuits**
 - short circuit = connection between 2 points of a circuit with resistance zero and imposing an electric voltage to the ends > 0
=> ~infinite current (Ohm law)
 - can burn components and Arduino as well
- Check the **max current** that be absorbed by a component
 - too much current => damages
- Check the **max current** to be absorbed by a pin of the MCU
- Check the component **polarity**
 - e.g. leds

FIRST ACTIVITIES IN LAB

- Step #0 - Hello world
- Step #1 - Blinking internal led
- Step #2 - Blinking external led
- Step #3 - Button-Led
- Step #4 - Tune
- Step #5 - Fading
- Step #6 - Counting (about interrupts)
- Step #7 - Event-driven blinking with Timers
- Step #8 - Deep sleep with interrupts
- Step #9 - Light sleep with timers
- Step #10 - Asynchronous serial: Serial echo
- Step #11 - Discovering I²C devices
- Step #12 - Using an I²C LCD

STEP #0 - HELLO WORLD

- **Description**

- sending “hello world” to the serial line every second, to be visualised by the serial monitor of the IDE running on the PC

- **Highlights**

- how the super loop works
- how to compile a program and upload on Arduino
- how to use the serial line and the serial monitor to print and visualise messages — good for debugging
- how to create a delay
- discipline about *.c *.cpp and *.h content
 - header files => only function/procedure signature declaration and var declarations (not definitions)
 - definitions/implementations only in *.c *.cpp sources

HELLO WORLD

```
void setup() {  
    Serial.begin(9600);  
}  
  
void loop() {  
    Serial.println("Hello, World!");  
    delay(1000);  
}
```

DETAILS: SERIAL OUTPUT

- Exploiting the serial connection with the PC to send messages that are displayed on PC window
 - useful for logging e for debugging
- In Wiring: Serial library
 - <http://arduino.cc/en/reference/serial>
 - class **Serial**
 - among the methods (static):
 - begin
 - » to initialize a serial communication session, specifying the baud rate
 - print/println
 - » to send string messages
 - write/read - per send/receive byte blocks

SERIAL MONITOR

- Feature provided by Arduino IDE to:
 - visualise messages sent through the serial connection
 - send messages as well
- It uses a configurable baud rate
 - which must be the same used on the Arduino side

STEP #1 - BLINKING INTERNAL LED

- **Description**

- The system must blink the on-board led periodically, with a 1000 ms period

- **Highlights**

- digital I/O - output
 - Wiring API
 - **pinMode, digitalWrite**
 - control loop

CODE

```
#define LED_PIN 13

void setup() {
    pinMode(LED_PIN, OUTPUT);
}

void loop() {
    digitalWrite(LED_PIN, HIGH);
    delay(1000);
    digitalWrite(LED_PIN, LOW);
    delay(1000);
}
```


BLINKING WITH SERIAL LOG

```
#define LED_PIN 13

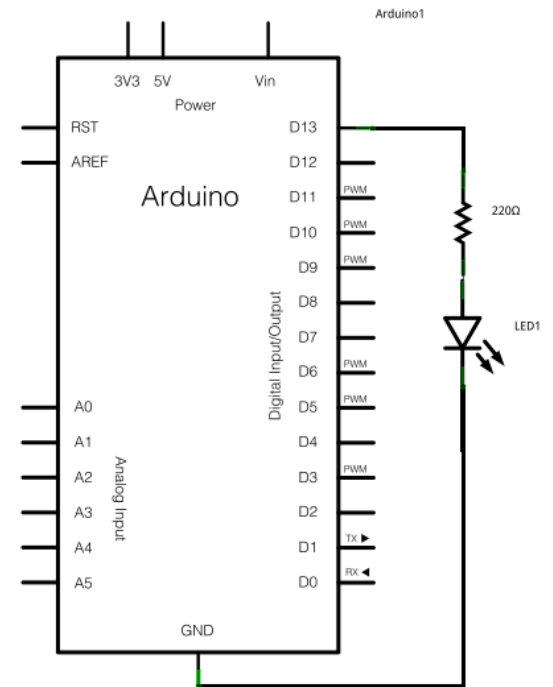
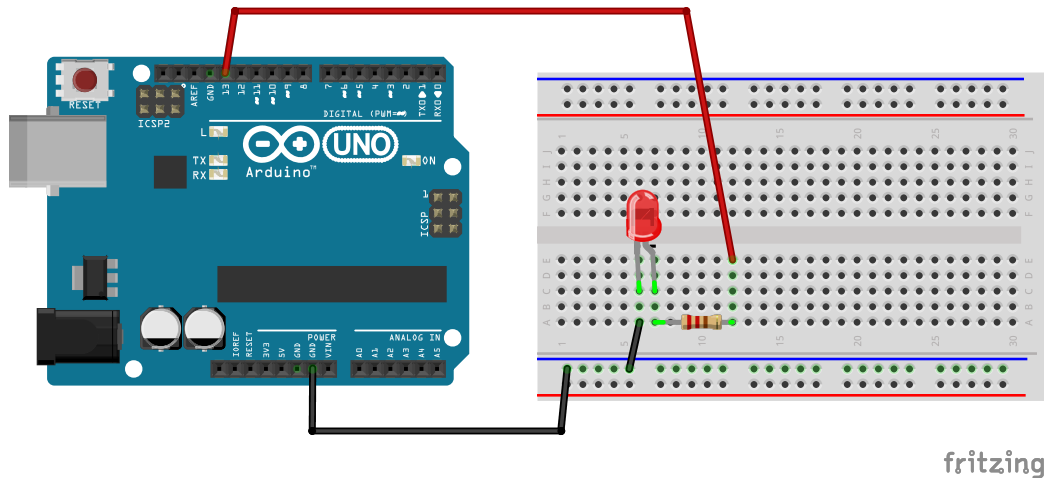
void setup() {
    pinMode(LED_PIN, OUTPUT);
    Serial.begin(9600);
}

void loop() {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("ON");
    delay(1000);
    digitalWrite(LED_PIN, LOW);
    Serial.println("OFF");
    delay(1000);
}
```

STEP #2: BLINKING AN EXTERNAL LED

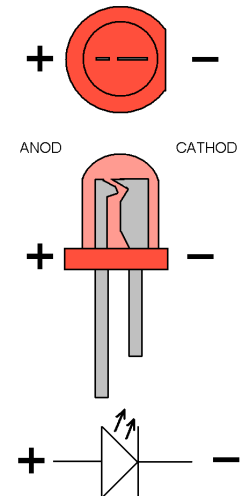
- **Description**
 - Blinking an external led, with a 1000 ms period
- HW
 - led, 220 Ohm resistor
- **Highlights**
 - HW
 - led - polarity
 - how to compute the resistance needed

CIRCUITS



LED

- LED (Light Emitting Diode)
 - optoelectronic devices — a diode that produces light when a current passes through
 - has a polarity
 - https://en.wikipedia.org/wiki/Light-emitting_diode
- It has an operating voltage, that depends on the color
 - examples
 - green: 2.0 V
 - red: 1.3 V
- Current consumption: 10..20 mA

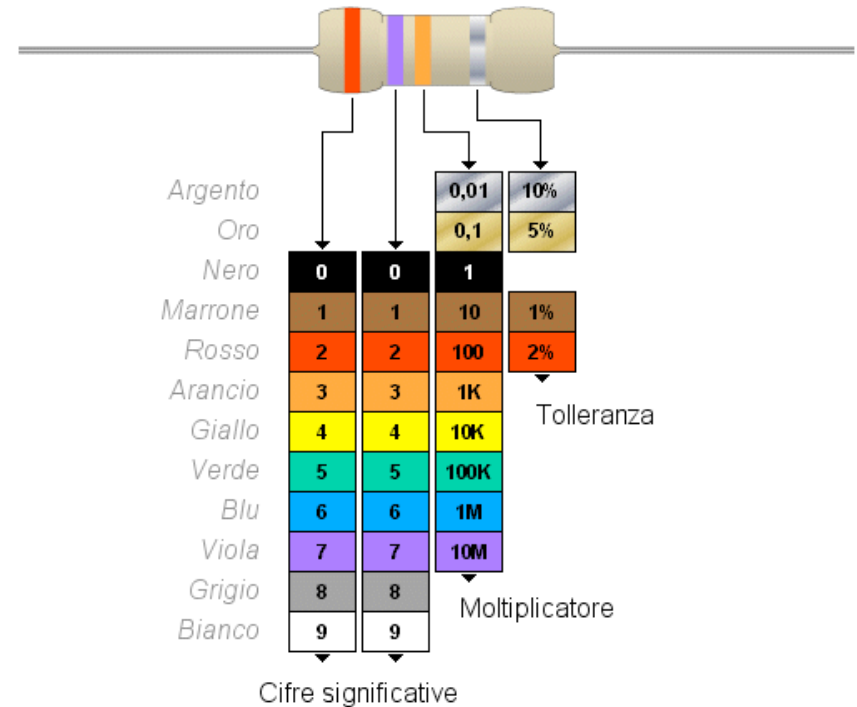


WHY CHOOSING THAT RESISTOR?

- The resistor makes it possible to have the right current quantity flowing through the LED, according its specs on the data sheet
 - <https://learn.adafruit.com/all-about-leds/the-led-datasheet>
- In order to work properly a LED should have a tension between + and - terminal (Forward Voltage) of about 2 Volt (between 1.8 and 3, depending on the color). 5 V are too much => going to break the LED if used for long time
- Given the 5V (when the pin is HIGH), we want 2V on the LED, then we need a resistor to make a voltage drop of 3 volts (3 - 2)
- Besides the tension, the LED datasheet reports that a LED works properly with a current (DC Forward Current) about 20 mA (30 mA max)
 - Arduino can supply max 40mA in output on a pin, so 20mA are ok
- Applying Ohm's law: the resistance R to be chosen to have a voltage drop of 3V and a current of 20mA is $R = V / I = 3 / 0.02 = 150 \text{ Ohm}$
 - the greater is the R value, less luminous is the LED
- We use a 220 Ohm resistor from our kit

220 OHM RESISTOR

- Resistance bands to compute the value of resistance
 - e.g. 4-bands resistors
- 220 Ohm => red-red-brown



STEP #3 - BUTTON-LED

- **Description**

- system with a tactile button + led. The led must be on when the button is pressed and off when the button is released

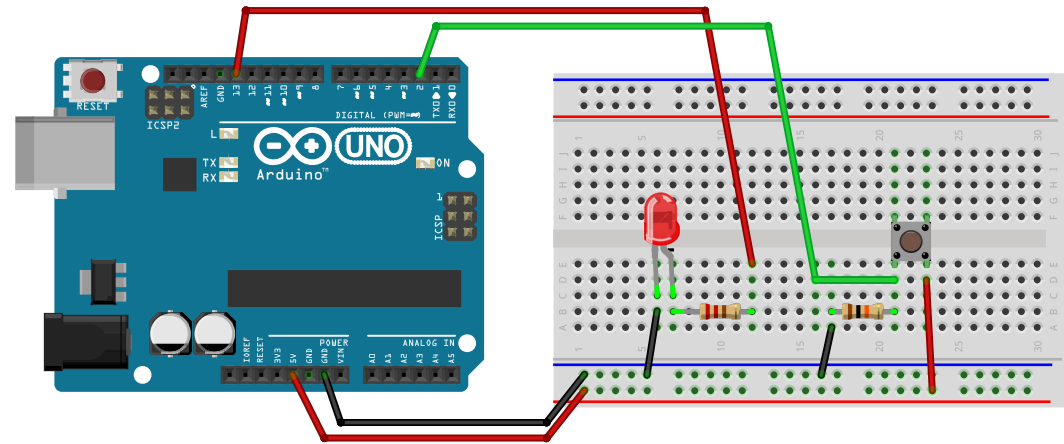
- **HW**

- led, 1 resistor 220 Ω , tactile button, 1 resistor 10 K Ω

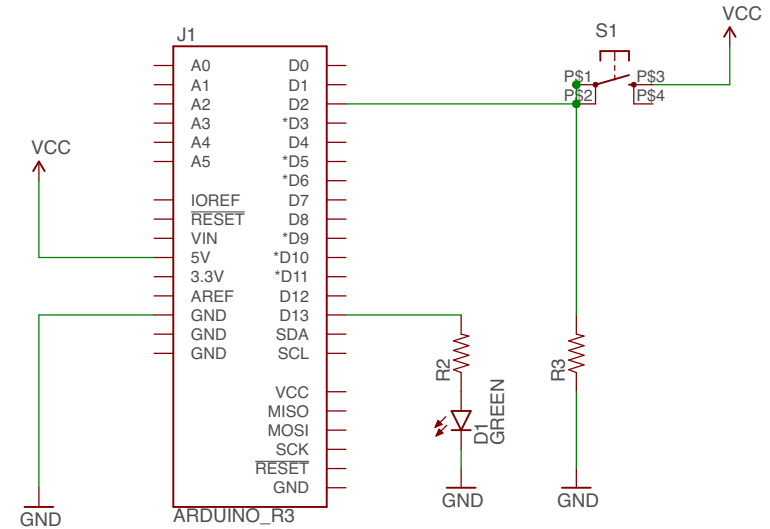
- **Highlights**

- digital I/O - input e output
 - Wiring API:
 - **digitalRead**
 - pull-down resistance

CIRCUITS



fritzing



PULL-DOWN RESISTOR

- pull-down resistor (10 K Ω) is used to make sure that — when the button is not pressed — the voltage level on the input pin would be LOW
 - “pull-down” because the signal is forced to LOW (GND)
 - “pull-up” instead => forced to HIGH (VCC)
- Big resistance value (es: 10 K Ω)
 - to minimise the current that flows in that part of the circuit

CODE

```
#define BUTTON_PIN 2
#define LED_PIN 13

void setup() {
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT);
  Serial.begin(9600);
}

void loop() {
  int buttonState = digitalRead(BUTTON_PIN);
  if (buttonState == HIGH) {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("ON");
  } else {
    digitalWrite(LED_PIN, LOW);
    Serial.println("OFF");
  }
}
```

ABOUT POLLING

- The approach used here to detect the button pressed is based on *polling*
 - *periodical read of the state of the pin*
- Problem
 - if the button is pressed too fast, it could happen that it is not detected..

STEP #4 - TUNE

- **Description**

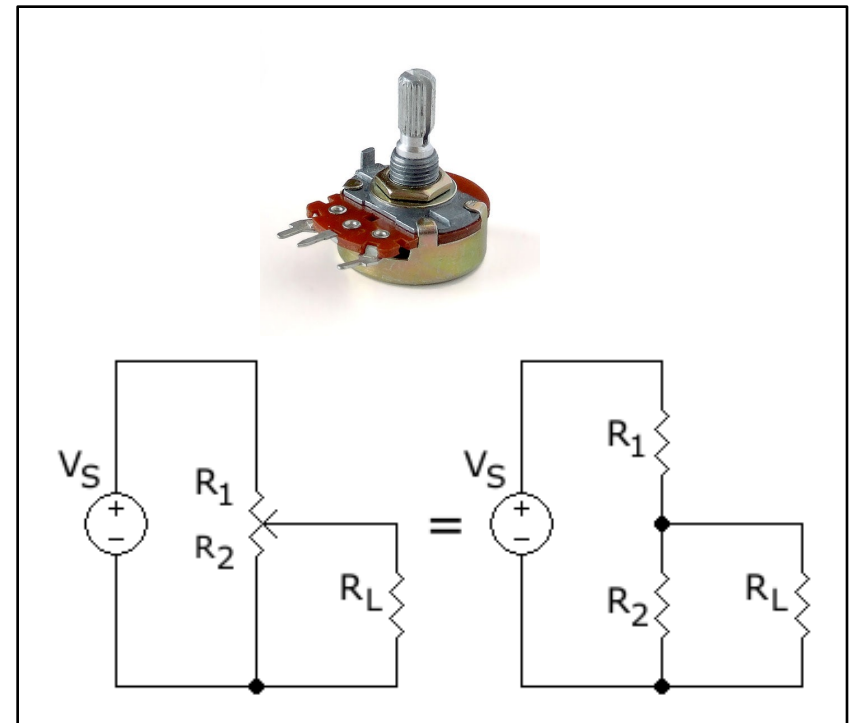
- system characterised by a potentiometer. It must log in output the current value of the potentiometer

- **HW**

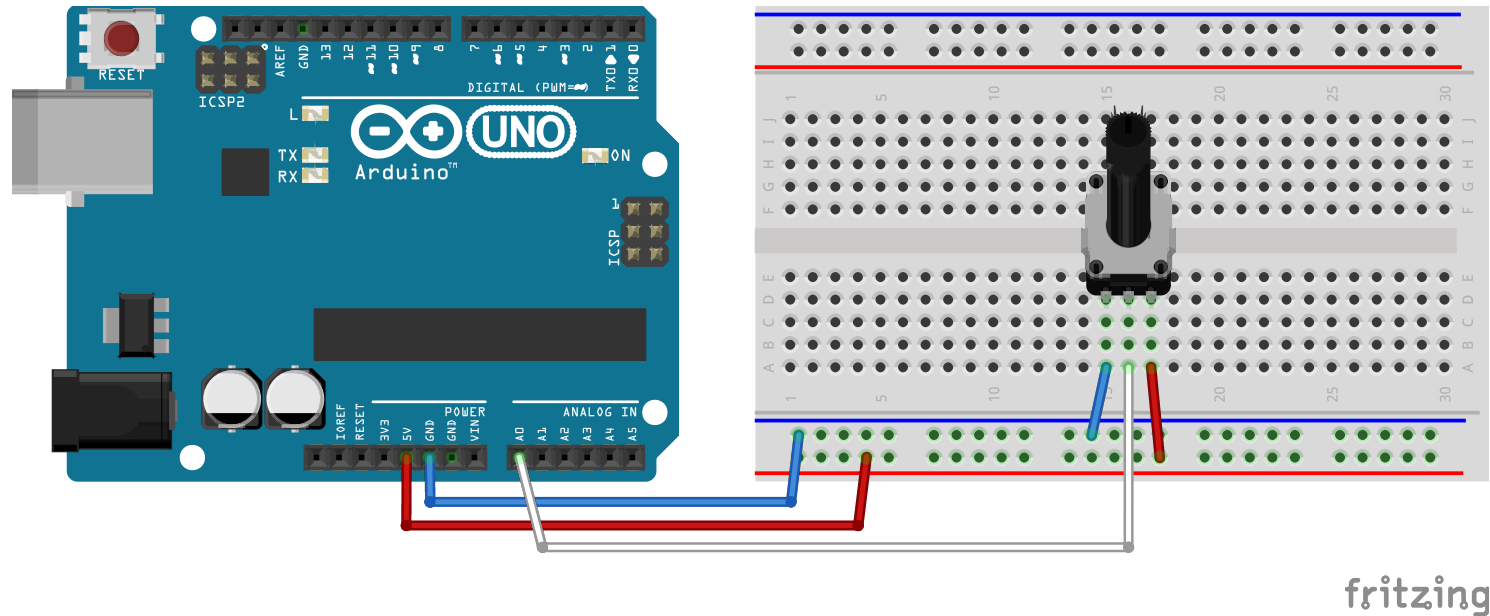
- a potentiometer

- **Highlights**

- analog pin reading
 - Wiring API
 - **analogRead**



CIRCUITS



CODE

```
#define POT_PIN A0

int current;

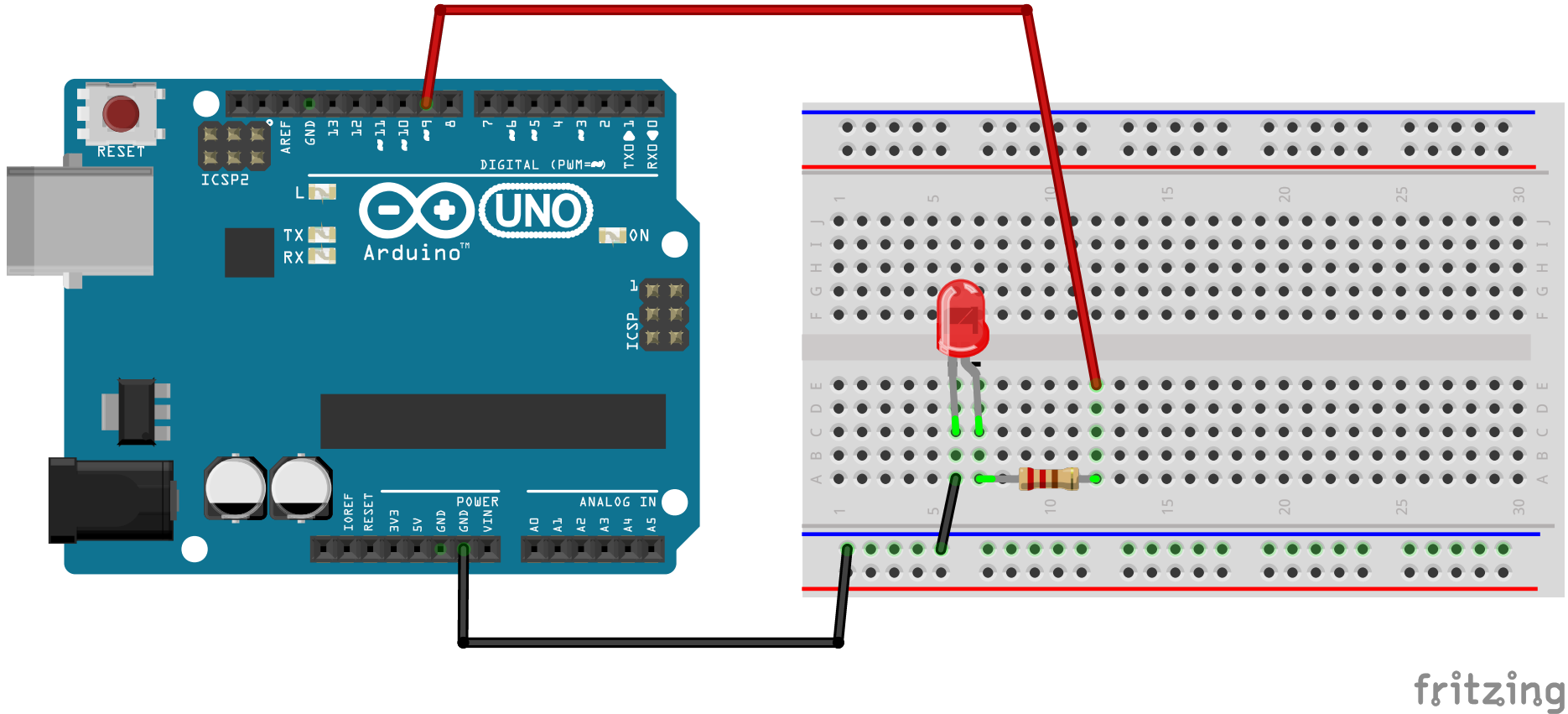
void setup() {
  Serial.begin(9600);
}

void loop() {
  int newValue = analogRead(POT_PIN);
  if (newValue != current){
    current = newValue;
    Serial.println(current);
  }
}
```

STEP #5 - FADING

- **Description**
 - system with a pulsing led, fading in and out
- HW
 - led, 220 Ω resistor
- **Highlights**
 - analog output and **PWM**
 - pins marked with ~
 - 3,5,6,9,10,11
 - Wiring API
 - **analogWrite**

CIRCUITS



CODE

```
// one supporting PWM
#define LED_PIN 9

int fadeAmount;
int currIntensity;

void setup(){
    currIntensity = 0;
    fadeAmount = 5;
    pinMode(LED_PIN, OUTPUT);
}

void loop(){
    analogWrite(LED_PIN, currIntensity);
    currIntensity = currIntensity + fadeAmount;
    if (currIntensity == 0 || currIntensity == 255) {
        fadeAmount = -fadeAmount ;
    }
    delay(20);
}
```

STEP #6 - “COUNTING”

- **Description**

- system that visualizes a count, which is incremented each time a button is pressed

- **HW**

- a tactile button, a 10 K Ω resistor

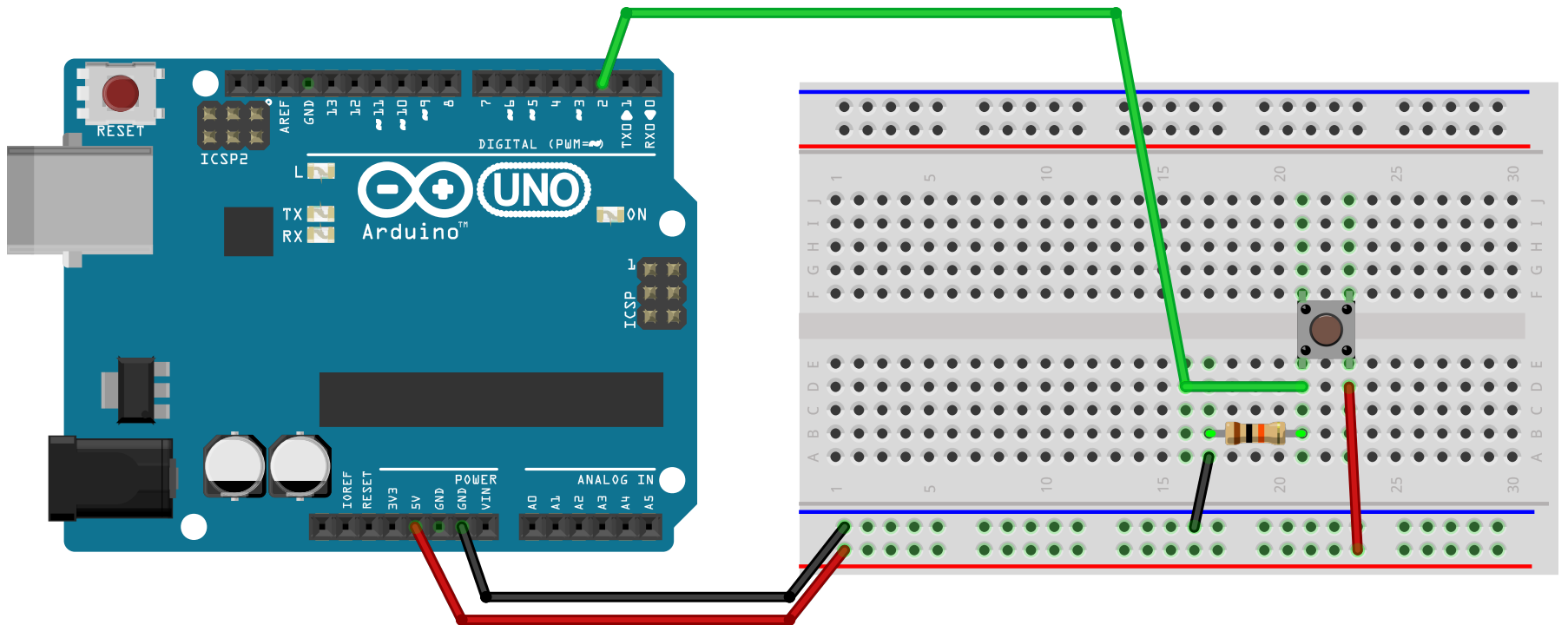
- **Highlights**

- interrupt

- **Problems**

- bouncing problem
- concurrency problems

CIRCUITS



fritzing

CODE

```
#define BUTTON_PIN 2
volatile int count = 0;
int prev = 0;

void setup(){
    Serial.begin(9600);
    attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), inc, RISING);
}

void loop(){
    noInterrupts();
    int current = count;
    interrupts();
    if (current != prev){
        Serial.println(current);
        prev = current;
    }
}

void inc(){
    count++;
}
```

BOUNCING PROBLEM

- **Bouncing** - physical phenomenon affecting tactile buttons
 - when pressed, there could be multiple internal mechanical rebounds before stabilising (in some milliseconds)
- These rebounds can generate a sequence of spurious interrupts
 - the counter is incremented more than 1 time
- Solutions (at the code level):
 - we can solve the problem by ignoring all the impulses that arrive shortly after the first one (e.g. 20-30 milliseconds)

ATOMICITY/CRITICAL SECTIONS

- In the control loop, the assignment of the count variable (that could be concurrently accessed by the interrupt handler too) must be atomic, i.e. should not be interruptible — to avoid race conditions
 - critical section
- To this purpose we can disable (and reenable) interrupts
 - **noInterrupts()/interrupts()**
- When interrupts are disabled, the MCU does is *not reactive* to external events (that are lost) => we should minimise the period of time in which interrupts are disabled
 - short-time interrupt handlers

FURTHER REMARKS

- Not all increments are printed...
- The reason is that logging is done asynchronously with respect to increment => there could be more than one increment between two logs

STEP #7 - EVENT-DRIVEN BLINKING

- **Description**
 - blinking the led by means of timer interrupts
- **HW**
 - led, 220 Ohm resistor
- **Highlights**
 - timer, library Timer1
 - <https://www.arduino.cc/reference/en/libraries/timerone/>
 - how to install a library on the IDE

CODE

```
#include "TimerOne.h"

#define LED_PIN 13

boolean flagState = false;

void blinky(){
    if (!flagState){
        digitalWrite(LED_PIN, HIGH);
    } else {
        digitalWrite(LED_PIN, LOW);
    }
    flagState = !flagState;
}

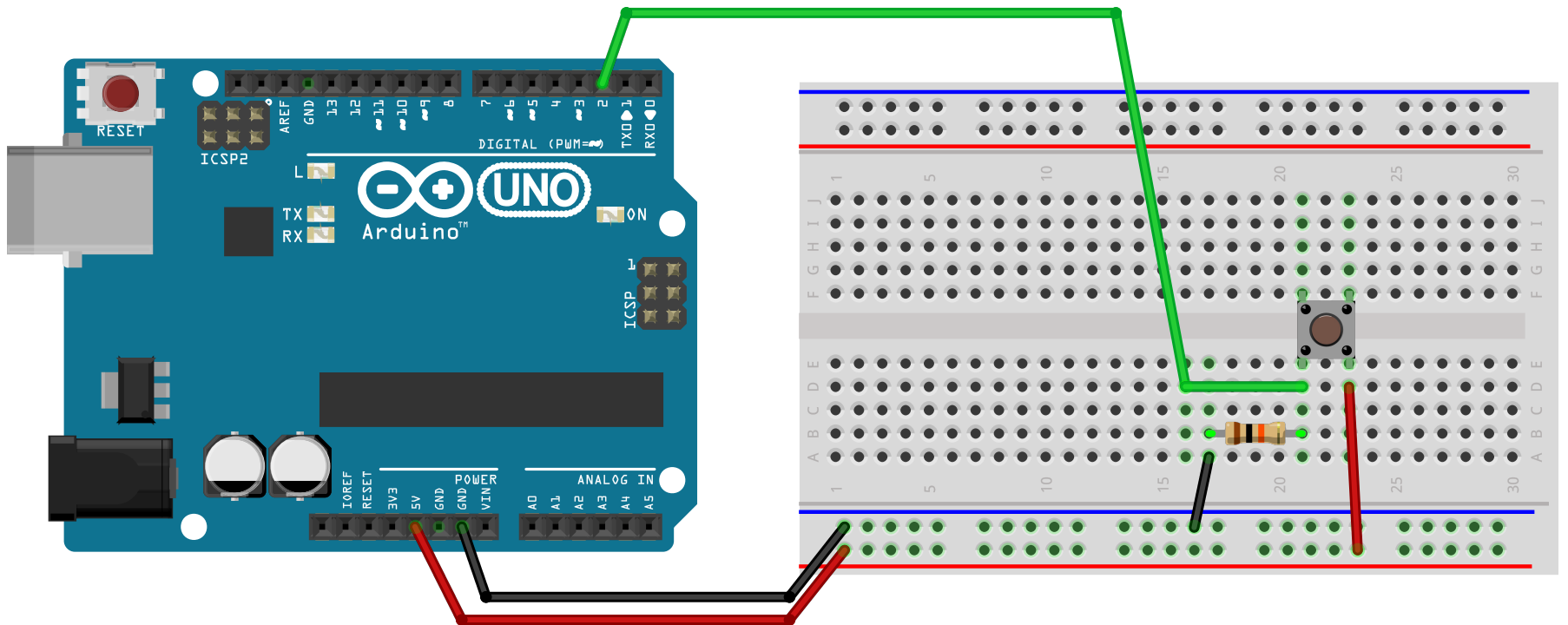
void setup()
{
    pinMode(LED_PIN, OUTPUT);
    /* set period timer 1000000 usec = 1 sec */
    Timer1.initialize(1000000);
    Timer1.attachInterrupt(blinky);
}

void loop(){}
}
```

STEP #8 - DEEP SLEEPING

- **Description**
 - testing sleep modes for low power consumption
 - Start with deep sleep
- HW
 - tactile button and 10 K Ω resistor
- **Highlights**
 - events that break the sleep and trigger restarting
 - time needed to restart

CIRCUITS



fritzing

CODE

```
#include <avr/sleep.h>

void wakeUp(){}

void setup(){
    Serial.begin(9600);
    pinMode(2,INPUT);
    attachInterrupt(digitalPinToInterrupt(2), wakeUp, RISING);
}

void loop(){
    Serial.println("POWER DOWN IN 1s ...");
    Serial.flush();
    delay(1000);

    set_sleep_mode(SLEEP_MODE_PWR_DOWN);
    sleep_enable();
    sleep_mode();
    /** The program will continue from here. **/
    Serial.println("WAKE UP");
    /* First thing to do is disable sleep. */
    sleep_disable();
}
```

STEP #9: LIGHT SLEEPING

- **Description**

- testing light sleep mode in which timers go on working

- **Highlights**

- disabling selectively the components that are not needed
 - useful approach for implementing schedulers (future modules)

CODE

```
#include <avr/sleep.h>
#include <avr/power.h>
#include "Timer.h"
Timer* timer;

void setup(){
    Serial.begin(9600);
    timer = new Timer(); /* timer 1 */
    timer->setupPeriod(5000);
}

void sleep(void)
{
    set_sleep_mode(SLEEP_MODE_IDLE);
    sleep_enable();
    power_adc_disable();
    power_spi_disable();
    power_timer0_disable(); // only timer 1
    power_timer2_disable(); // on
    power_twi_disable();
    sleep_mode();
    /* back */
    sleep_disable();
    power_all_enable();
}
```

```
void loop(){
    Serial.println("GOING TO SLEEP");
    Serial.flush();
    sleep();
    Serial.println("WOKE UP");
}
```

STEP #10 - SERIAL ECHO

- **Description**
 - reading messages sent from the serial line and send them back
- **Highlights**
 - library Serial

CODE

```
void setup()
{
  Serial.begin(9600);
}

void loop()
{
  if (Serial.available()){
    char data = Serial.read();
    Serial.print(data);
  }
}
```


STEP #11 - DISCOVERING I²C DEVICES

- **Description**
 - Scanning the I²C bus for discovering the devices
- **Highlights**
 - Wire.h library
 - I²C device used: an LCD

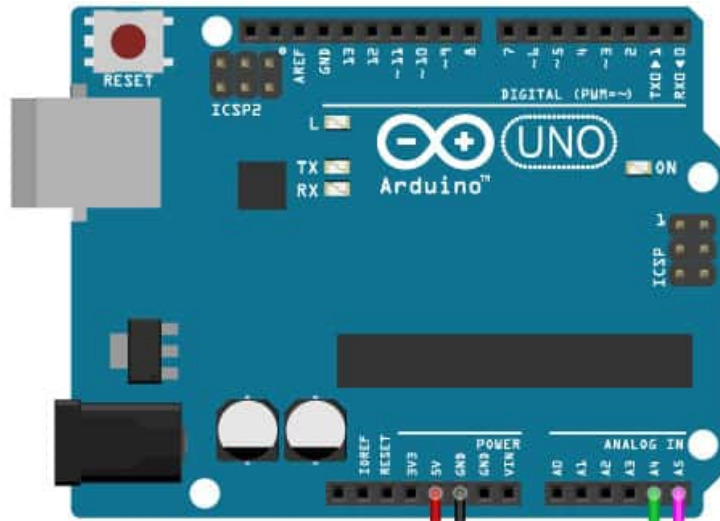
DISPLAY LCD

- Display based on the optical properties of liquid crystals
 - trapped between two glass surfaces, equipped with many electrical contacts that can generate an electric field affecting the liquid
 - each electric contact control a small portion of the panel that corresponds to a pixel (or a subpixel in a color screen)
- Small power consumption
 - very good for embedded systems

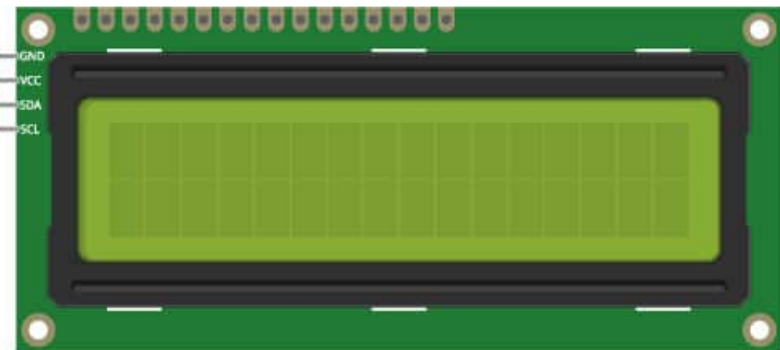
PARALLEL LCD SCREEN

- A very common example is the parallel LCD screen 16x2 (16 columns, 2 rows), characterised by a single line of 16 pins
- All parallel LCD screen have the same pinout and can be connected to Arduino in different ways:
 - 4 pins or 8 pins
 - 8 pins mode: 6 pins must be connected to digital pins + 2 for VCC and GND
 - 6 pins mode: 4 pins are used to send the data and the other 2 for control
 - **via I2C**
 - **es: <http://www.robot-italy.com/it/display-lcd-16-x-2-blu-i2c.html>**
- Programming
 - LiquidCrystal Library provided with Arduino IDE (with I2C version)
 - many functionalities
 - cursor blinking, scrolling, customised chars creation, etc.

CONNECTING A I²C-BASED LCD TO BE DISCOVERED



- Arduino is the primary (master) device
- The LCD is a secondary (slave)



fritzing

CODE

```
#include <Wire.h>

void setup() {
  Wire.begin();
  Serial.begin(9600);
  while (!Serial);
  Serial.println("\nI2C Scanner");
}

void loop() {
  byte error, address; int nDevices;
  Serial.println("Scanning...");
  nDevices = 0;
  for (address = 1; address < 127; address++ ) {
    Wire.beginTransmission(address);
    error = Wire.endTransmission();
    if (error == 0) {
      Serial.print("I2C device found at address 0x");
      if (address < 16) Serial.print("0");
      Serial.print(address, HEX);
      Serial.println(" !");
      nDevices++;
    } else if (error == 4) {
      Serial.print("Unknown error at address 0x");
      if (address < 16)
        Serial.print("0");
      Serial.println(address, HEX);
    }
  }
}
...
```

```
...

if (nDevices == 0)
  Serial.println("No I2C devices found\n");
else
  Serial.println("done\n");

delay(5000);
}
```

STEP #12 - USING AN I²C LCD

- **Description**
 - Using a I²C-based LCD
- **Highlights**
 - Abstracting from I²C specific code by means of device-specific libraries

CODE

```
#include <LiquidCrystal_I2C.h>

/* Wiring: SDA => A4, SCL => A5 */
/* I2C address of the LCD: 0x27 */
/* Number of columns: 20 rows: 4 */

LiquidCrystal_I2C lcd = LiquidCrystal_I2C(0x27,20,4);

void setup() {
    lcd.init();
    lcd.backlight();
}

void loop() {
    /* Set the cursor on the third column and first row. */
    lcd.setCursor(2, 1);
    lcd.print("Hello World!");
}
```